Team GraphTheWorld : Link Prediction in an Actor Co-occurrence Network

Henri Bonamy Ali Dor Elora Drouilhet Zaynab Raounak

🏆 1st Place — Public AUC: **0.88822**

1. Feature Engineering — 73 Scalar Features from 3 Data Sources

We extract features from graph topology (A), 932 binary keywords (`node_information.csv`), and test-set structure. Each family targets a specific co-occurrence signal.

Family	#	What it captures	Key ref.	ΔAUC
Graph topology	14	Local overlap (CN, Jaccard, Adamic-Adar, Res. Alloc.), hub structure, paths of length 3 via A^3	[1–3]	baseline
Text similarity	9	Keyword overlap: cosine, TF-IDF, Jaccard, asymmetric overlap from `node_information.csv`	—	+0.019
Neighb. text	4	1-hop GCN-style aggregation of neighbors' TF-IDF vectors [6]	[6]	+0.006
Node transductive	6	How often each node appears in train/test pairs (meta-information)	—	+0.011
Pair transductive	15	Shared test partners, transductive triangles, AA/RA in test-partner space	—	+0.005
Community	10	Louvain [4], spectral [5], text-weighted Louvain, multi-seed consensus	[4, 5]	+0.012
SVD role sim.	3	Rank-4 SVD cosine + Hadamard axis products [7, 8]	[7, 8]	+0.003
Total	**73**			**0.888**

All 73 features with motivation, source reference, and incremental AUC contribution.

Key Design Decisions

Graph: Paths of length 3. $A^3[u,v]$ counts indirect walks via sparse matrix multiplication. For training edges, we subtract $\deg(u)+\deg(v)-1$ spurious walks from the direct edge to prevent leakage.

Text: Neighborhood aggregation. We average each node's neighbors' TF-IDF vectors (from `node_information.csv`) and compute pairwise cosines—a manual 1-hop GCN without neural training. Leakage-corrected by excluding direct neighbor contributions.

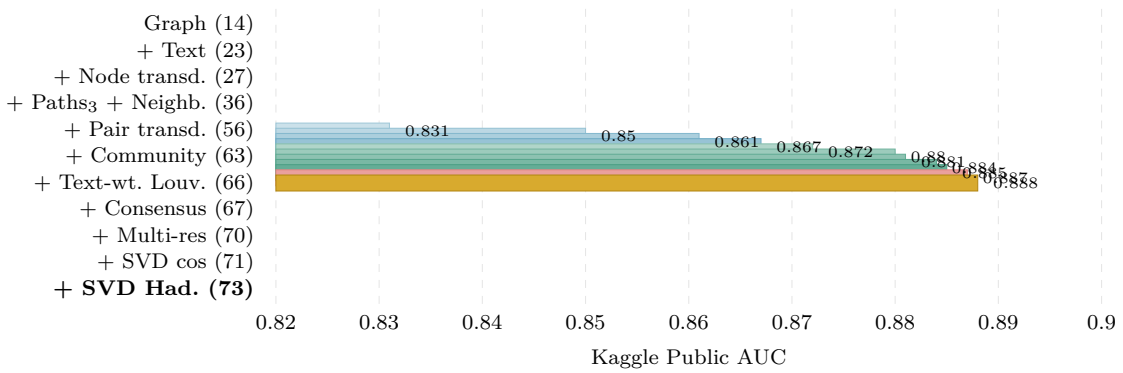
Transductive: Shared test partners. For pair (u,v): $|\text{test_partners}(u) \cap \text{test_partners}(v)|$. Positive pairs show **12**× more shared partners and **23**× more transductive triangles. Uses only test-set *structure*, never labels.

Community: Multi-seed consensus. Louvain at 20 seeds; co-clustering fraction as $[0,1]$ score replaces noisy binary membership. **+0.00352 Kaggle**—biggest single-feature gain.

SVD: Hadamard products. Cosine collapses all axes into one scalar; element-wise products ($\mathbf{e}_u \odot \mathbf{e}_v$) let the booster split on each SVD axis independently. Axis 4 alone: +0.00084. This captures *role similarity* (same connection pattern) vs. Louvain's *cluster membership*.

Feature selection. Strict **one-at-a-time forward selection** on 30-seed blend OOF AUC. Bundling features hurt (v26c: 10 features, −0.001).

Ablation Progression



Cumulative AUC. Red: SVD breakthrough. Failed: GCN (0.670), Node2Vec (0.770), SEAL (0.701), TabPFN (no-op).

2. Model Tuning & Comparison

Models Explored

We evaluated five model families on the same 73-feature set. The table below summarizes results; detailed analysis of each approach follows.

Model	Kaggle	OOF	Analysis
GCN (2-layer, 64h)	0.670	—	10k samples insufficient for neural params; embeddings lose to scalar heuristics
Logistic Regression	0.831	0.841	Strong linear baseline; cannot capture feature interactions
Random Forest	0.845	0.862	Captures interactions but high variance on small data
CatBoost [10]	0.880	0.910	Ordered boosting reduces prediction shift; best single model
HGB+CatBoost [9]	**0.888**	**0.914**	Complementary split strategies; rank-averaged ensemble

Final Models: HistGradientBoosting & CatBoost

Our two final models are both gradient-boosted decision trees (GBDT), chosen for their ability to capture complex feature interactions while remaining resistant to overfitting on small datasets through regularization.

HistGradientBoosting (HGB) [9] is scikit-learn's implementation inspired by LightGBM. It discretizes continuous features into 256 histogram bins before finding optimal splits, which acts as implicit regularization and dramatically accelerates training. HGB uses *leaf-wise* (best-first) tree growth: at each step, it splits the leaf with the highest loss reduction, producing asymmetric trees that focus capacity on hard examples. We set `min_samples_leaf`=40 to prevent overly specific leaves on our 10k training set.

CatBoost [10] uses *ordered boosting*, a permutation-based approach that computes gradient estimates using only preceding observations in a random permutation. This avoids the *prediction shift* problem [10]—the bias that arises when the same data points used to estimate gradients are also used to fit the next tree. On our small dataset, this provides a measurable advantage: CatBoost alone (0.880) outperforms HGB alone (0.878). CatBoost grows *symmetric* (level-wise) trees where all nodes at a given depth use the same split feature and threshold, acting as strong structural regularization.

Why both? HGB and CatBoost explore different regions of the hypothesis space due to their distinct tree-growth strategies (leaf-wise vs. symmetric) and gradient estimation procedures (standard vs. ordered). Their prediction errors are partially uncorrelated—HGB tends to rank certain transductive features higher, while CatBoost relies more on community/SVD features. Blending them captures complementary patterns.

Hyperparameter Tuning via 5-Fold Stratified CV

For both models, we performed grid search over the parameters below, evaluated on 5-fold stratified CV AUC:

Parameter	Grid searched	HGB best	CatBoost best
`learning_rate`	{0.01, 0.03, 0.05, 0.1}	0.05	0.05
`max_depth`	{2, 3, 4, 5}	3	3
`iterations`	{200, 300, 400, 500}	400	300
`min_samples_leaf`	{20, 30, 40, 60}	40	—
`l2_regularization`	{0.01, 0.1, 1, 10}	0.1	10

Both models converged to shallow trees (`depth`=3) and moderate learning rates. Depth≥5 overfitted consistently (−0.003 OOF).

The convergence of both models to `depth`=3 reflects the dataset's small size: deeper trees memorize training patterns (especially graph features that leak through positive edges in the adjacency). CatBoost requires stronger L2 regularization (10 vs. 0.1) because its symmetric trees have more uniform capacity across the feature space.

Ensemble Strategy & Rank Averaging

Since AUC depends only on the *ranking* of predictions, not their magnitudes, we use **rank averaging**: each model's raw probabilities are converted to ranks $\in [0, 1]$ (via percentile transform), then averaged. This eliminates calibration mismatches—HGB outputs tend to be more extreme (closer to 0/1) while CatBoost produces more conservative probabilities. Direct probability averaging would let the more extreme model dominate; rank averaging gives equal weight to each model's ordering.

Each model is trained with **30 random seeds**, producing 30 HGB + 30 CatBoost predictions (60 total). Averaging over seeds reduces variance: single-seed HGB std ≈ 0.003 on OOF; 30-seed std drops to ≈ 0.0005. The 60 rank vectors are averaged into one final prediction.

Self-Training

We apply one conservative round of self-training: predict test pairs with the current ensemble, add ∼332 pairs with $p \geq 0.95$ as pseudo-edges, then **recompute all graph-based features** (CN, Adamic-Adar, paths, community, SVD) on the augmented adjacency. The enriched features improve graph heuristics for nodes that were previously isolated. We then retrain the ensemble from scratch on the original labels with the updated features.

Multiple rounds ($\geq$2) or lower thresholds ($\leq$0.90) consistently degraded performance by introducing noisy pseudo-edges that corrupt the graph structure—particularly damaging for community detection and SVD, which are sensitive to spurious connections.

Overfitting Analysis

OOF–Kaggle gap. Our OOF AUC (0.914) exceeds Kaggle (0.888) by 0.026. Two identified sources: **(1)** Graph feature leakage: positive training edges remain in the adjacency during CV, inflating CN/AA scores for positive pairs. **(2)** Transductive leakage: node/pair counts span train+test, so CV folds see test-set structure that wouldn't exist in a purely inductive setting. Mitigations: 30-seed averaging, one-at-a-time feature ablation validated on *Kaggle only*, and selecting both final submissions based on public leaderboard scores—never OOF. This discipline should help generalization to the private test partition.

Final Submissions

Submission	Feat.	Public AUC	Innovation
`v26R`	73	**0.88822**	SVD Hadamard axis products
`v26V`	77	0.88683	Consensus SVD via edge-dropout variance

v26R adds `svd3_cos` and `svd4_had3` to the v26L base. **v26V** replaces the single SVD with 20 subsampled copies (5% edge dropout), adding mean/variance features that measure embedding robustness. It scored lower because rank-4 SVD is already stable on this graph, limiting the variance signal. We selected v26R as primary and v26V as a hedge with a structurally different feature philosophy.

3. Code & Reproducibility

Full code: https://github.com/helloelora/graph-data-challenge

File	Contents
`main.ipynb`	End-to-end pipeline with inline comments referencing report sections
`feature_engineering/`	Modular functions per family (graph, text, transductive, community, SVD)
`models/`	HGB/CatBoost wrappers: CV, seed averaging, rank blending
`config.py`	All hyperparameters centralized for reproducibility

References

[1] D. Liben-Nowell, J. Kleinberg. The link-prediction problem for social networks. *JASIST*, 58(7), 2007.

[2] L. Adamic, E. Adar. Friends and neighbors on the web. *Social Networks*, 25(3), 2003.

[3] L. Lü, T. Zhou. Link prediction in complex networks: A survey. *Physica A*, 390(6), 2011.

[4] V. Blondel et al. Fast unfolding of communities in large networks. *J. Stat. Mech.*, 2008.

[5] J. Kunegis, A. Lommatzsch. Learning spectral graph transformations for link prediction. *ICML*, 2009.

[6] D. Zhou et al. Learning with local and global consistency. *NeurIPS*, 2004.

[7] S. Deerwester et al. Indexing by latent semantic analysis. *JASIST*, 41(6), 1990.

[8] A. Grover, J. Leskovec. node2vec: Scalable feature learning for networks. *KDD*, 2016.

[9] G. Ke et al. LightGBM: A highly efficient gradient boosting decision tree. *NeurIPS*, 2017.

[10] L. Prokhorenkova et al. CatBoost: Unbiased boosting with categorical features. *NeurIPS*, 2018.